
Beispiele für mögliche Prüfungsaufgaben im Fach Computational Engineering

Hinweise zu diesem Dokument:

- In Ihrer Prüfung werden **drei** Aufgaben von dem hier präsentierten Typ vorkommen.
- Die Aufgaben können aus allen Kapiteln stammen, die in der Vorlesung und Übung behandelt wurden. Eine Eingrenzung hierzu gibt es nicht.
- Die hier präsentierten Aufgaben sind zufällig ausgewählt und die Auswahl stellt ebenfalls keine Eingrenzung dar.
- Alle Aufgaben werden in etwa die gleiche Punktzahl haben.
- Für eine 100%-Wertung in dieser Prüfung benötigen Sie ca. 80% der maximalen Gesamtpunktzahl. Den genauen Anteil lege ich nach der Aufwändigkeit der konkret ausgewählten Aufgaben fest.
- Bestanden haben Sie, wenn Sie aus allen drei Aufgaben 50% der Punktzahl erreichen, welche der 100%-Marke entspricht.
- Die Aufgaben in der Prüfung orientieren sich an den Übungen, die während des Semesters bearbeitet wurden. Sie dürfen sämtliche Codes aus dem Semester als Hilfe benutzen.
- Alle Bestandteile der von Ihnen abgegebenen Programme, die nicht zur Lösung erforderlich sind, also insbesondere unnötige Copy and Paste Rückstände, führen u.U. zu Punktabzug.

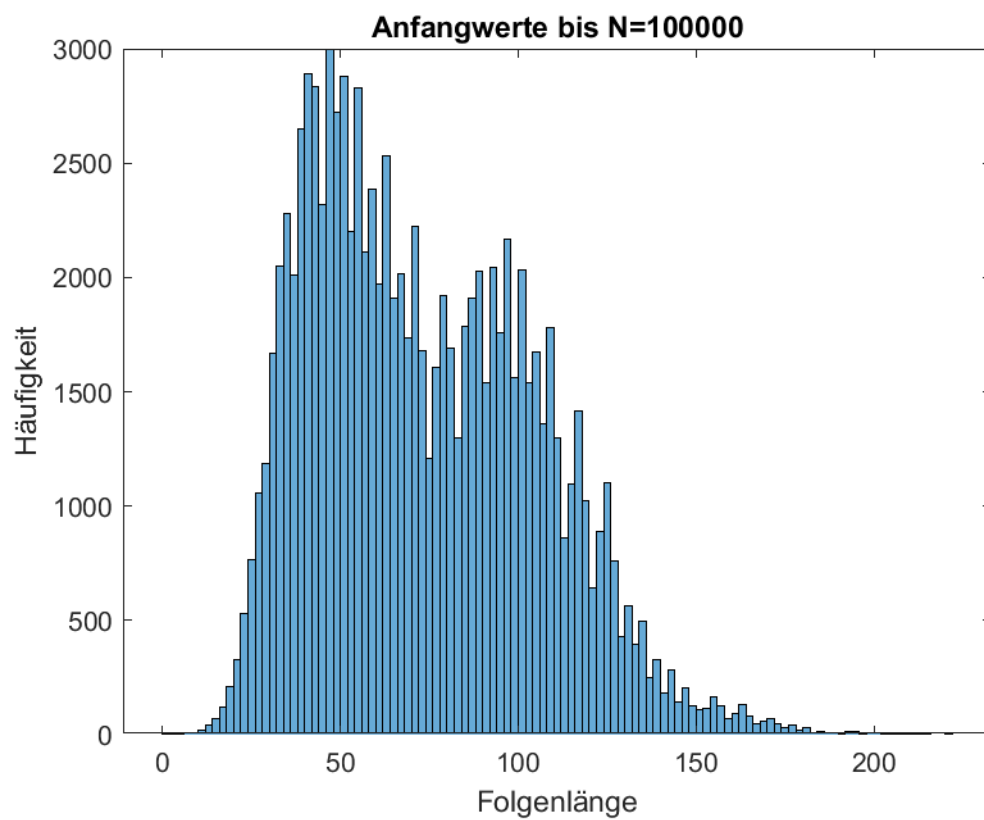
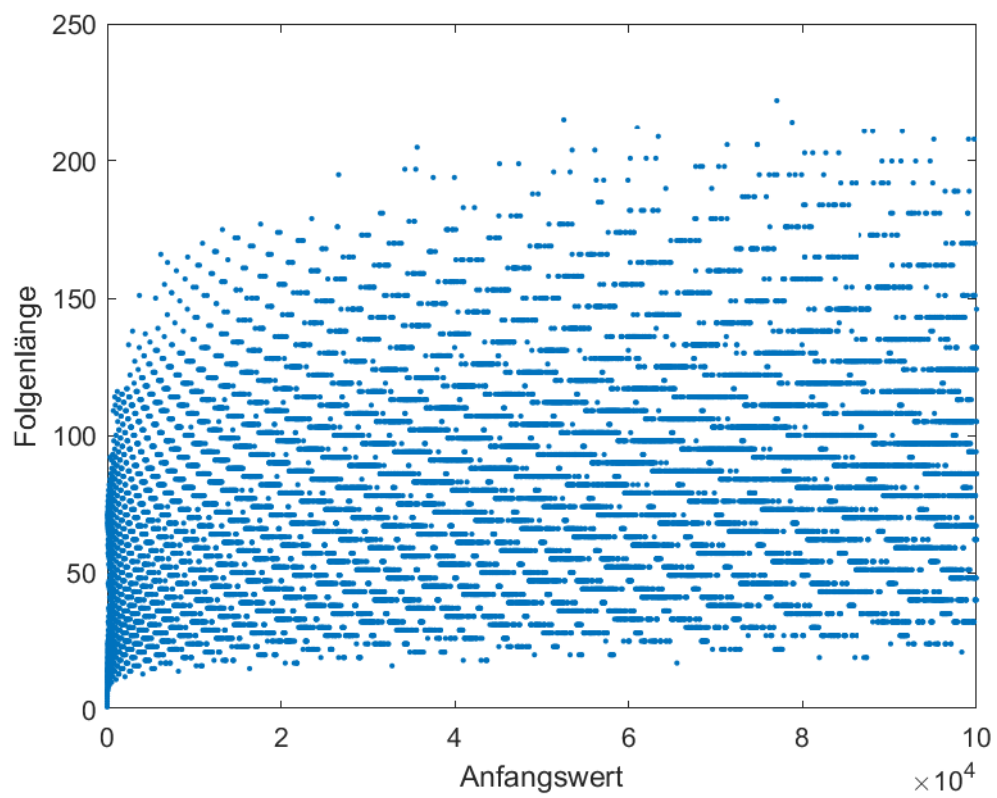
Beispiel-Aufgabe 1

Gegeben Sei ein Startwert $n_0 \in \mathbb{N}$ und folgende Iterationsvorschrift

$$n_{k+1} = \begin{cases} \frac{n_k}{2} & \text{wenn } n_k \text{ gerade} \\ \frac{3n_k+1}{2} & \text{wenn } n_k \text{ ungerade} \end{cases}$$

- Schreiben Sie eine Matlab Funktion, die als Übergabaparameter den Startwert n_0 erhält.
- Innerhalb der Funktion wird die Iteration, ausgehend vom übergebenen Startwert n_0 , so lange durchgeführt, bis für einen Wert $K \in \mathbb{N}$ gilt $n_K = 1$.
- Die Anzahl der Iterationen K bis der Wert $n_K = 1$ erreicht wurde, ist der Rückgabewert der Funktion.
- Die Iteration bricht ab, wenn die Iterationslänge eine maximale Zahl von $K_{\max} = 10^6$ überschreitet
- Rufen Sie die Funktion dann für Startwerte $n_0 = 1, 2, 3, \dots, 10^5$ auf und speichern Sie die zugehörigen Iterationslängen $K(n_0)$ in einem Vektor ab.
- Fertigen Sie zwei Graphiken an. In der ersten Stellen Sie die Iterationslänge K in Abhängigkeit des Startwertes n_0 dar. In der zweiten Graphik erstellen Sie ein Histogramm der Iterationslängen. Die Achsen in beiden Graphiken sind zu beschriften.

```
N=10^5;
laengen=zeros(1,N);
for n=1:N
    laengen(n) = fun_collatz(n);
end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
function out = fun_collatz(n)
    count=1;
    Kmax=10^6;
    while n~=1 && count<=Kmax
        if mod(n,2) == 0
            n = n/2;
        else
            n = (3*n + 1)/2;
        end
        count = count + 1;
    end
    out = count;
end
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
figure plot(laengen, '.')
xlabel('Anfangswert') ylabel('Folgenlänge')
figure
histogram(laengen)
xlabel('Folgenlänge') ylabel('Häufigkeit')
title(['Anfangswerte bis N=' num2str(N)])
```



Beispiel-Aufgabe 2

Schreiben Sie eine Matlab-Funktion, die als Übergabewerte eine quadratische Koeffizientenmatrix $\mathbf{A}$ und den Vektor der rechten Seiten $\vec{b}$ eines quadratischen linearen Gleichungssystems $\mathbf{A} \cdot \vec{x} = \vec{b}$, sowie einen Startvektor $\vec{x}_0$ und eine Toleranzschwelle δ erhält. Innerhalb der Funktion lösen Sie das lineare Gleichungssystem dann iterativ, und zwar in jeder Iteration sowohl mit dem Gesamtschritt-Verfahren, als auch dem Einzelschritt-Verfahren. Die Iteration wird nach dem n ten Iterationsschritt abgebrochen, wenn die Abbruchbedingung

$$|\vec{x}_{\text{Gesamtschritt}}^{(n)} - \vec{x}_{\text{Einzelschritt}}^{(n)}| < \delta$$

erfüllt ist. Die Funktion gibt dann den Punkt $\vec{x}_{\text{Einzelschritt}}^{(n)}$ und die Iterationszahl n nach der abgebrochen wurde, als Ausgabe zurück. Sie dürfen die Matlab Funktion `inv` benutzen. Schützen Sie Ihre Funktion gegen mindestens zwei Fehleingaben. Als Testobjekte können Sie

$$\mathbf{A} = \begin{pmatrix} 8 & -4 & -2 \\ 3 & -6 & 5 \\ 5 & -5 & 7 \end{pmatrix}, \quad \vec{b} = \begin{pmatrix} -6 \\ -3 \\ 11 \end{pmatrix} \quad \vec{x}_0 = \begin{pmatrix} 0 \\ 0 \\ 0 \end{pmatrix} \quad \text{und} \quad \delta = 10^{-3}$$

verwenden.

```
A = [-5, 2,-4; 3, 6,-5; -1, 8,-9]; b = [0;-8;-10];
x0 = [10,10,10];
delta = 10^(-3);
[x,n] = fun_A2_A(A,b,x0,delta);
function [out1,out2] = fun_A2_A(A,b,x0,delta)
    [z,s]=size(A);
    if z~=s
        error('Die Koeffizientenmatrix muss quadratisch sein!')
    end
    if length(x0)~=z
        error('Der Startpunkt hat nicht die korrekte Dimension!')
    end
    D = diag(diag(A)); L = tril(A)-D; R = triu(A)-D;
    invD = inv(D); invDL = inv(D+L);
    xJ(:,1) = x0; xG(:,1) = x0;
    xJ(:,2) = -invD*(L+R)*xJ(:,1)+invD*b;
    xG(:,2) = -invDL*R*xG(:,1)+invDL*b;
    test = norm(xJ(:,2)-xG(:,2));
    k = 2;
    while test > delta
        xJ(:,k+1) = -invD*(L+R)*xJ(:,k)+invD*b;
        xG(:,k+1) = -invDL*R*xG(:,k)+invDL*b;
        test = norm(xJ(:,k+1)-xG(:,k+1));
        k=k+1;
    end
    out1 = xG(:,end);
    out2 = k;
end
```

Beispiel-Aufgabe 3

Gegeben ist die abschnittsweise definierte periodische Funktion

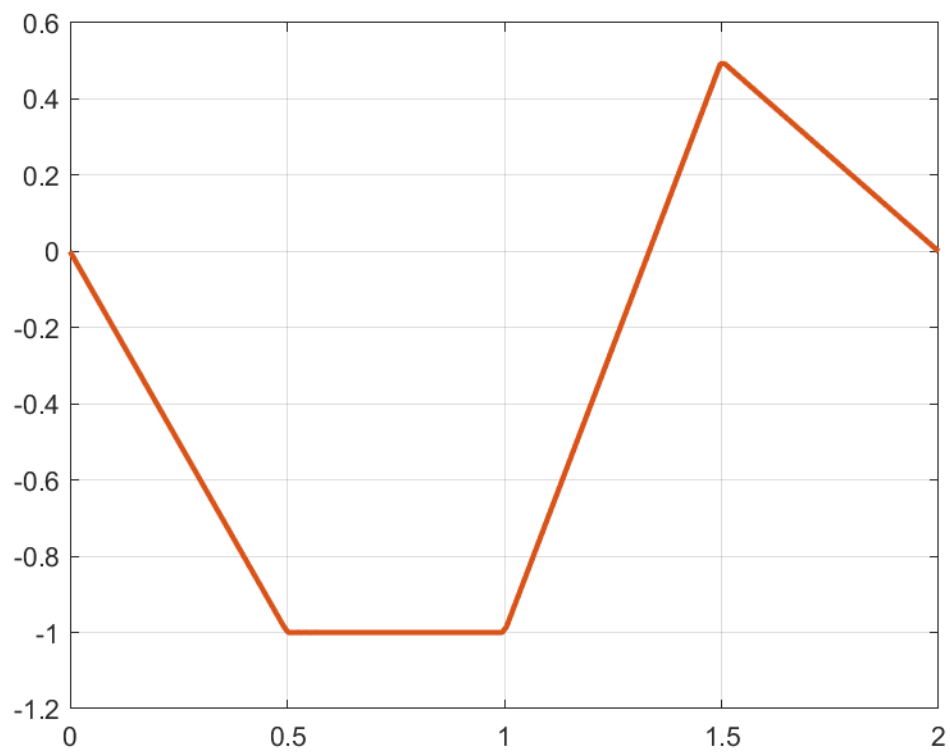
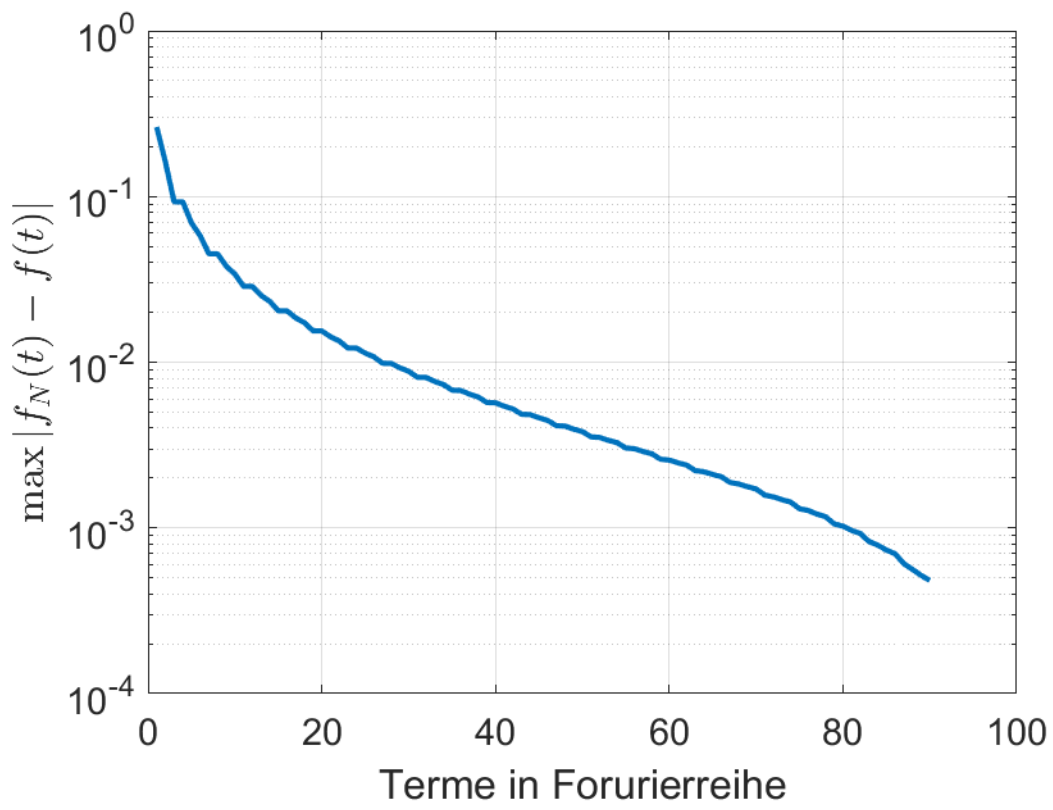
$$f(t) = \begin{cases} -2t & 0.0 \leq t < 0.5 \\ -1 & 0.5 \leq t < 1.0 \\ 3t - 4 & 1.0 \leq t < 1.5 \\ -t + 2 & 1.5 \leq t \leq 2.0 \end{cases} \quad \text{mit} \quad f(t+2) = f(t)$$

Geben Sie ein Matlab-Skript an, in welchem Sie die Fourierreihe $f_N(t)$ der Funktion auf dem Intervall $t \in [0, 2]$ berechnen. Teilen Sie das t -Intervall für diese Berechnung in mindestens 200 Punkte auf. Die Anzahl N der Terme in der Fourierreihe ist dabei so zu wählen, dass für die Abweichung Δ von Fourierreihe und Funktion, welche wie folgt definiert wird

$$\Delta = \max_{t \in [0, 2]} |f_N(t) - f(t)|$$

eine Toleranz von $\Delta < 0.5 \cdot 10^{-3}$ eingehalten ist. Stellen Sie am Ende in einer Graphik mit halblogarithmischer Skala, das Verhalten von Δ mit steigender Anzahl von berücksichtigten Termen in der Fourierreihe dar. In einer zweiten Graphik stellen Sie die Funktion und die Fourierreihe zusammen dar.

```
TOL = 0.5*10^(-3); omega = pi;
t = linspace(0,2,200); s = zeros(1,length(t));
for ii = 1:length(s)
    s(ii) = signal(t(ii));
end
a0 = trapz(t,s);
sN = (1/2)*a0.*ones(1,length(t));
delta = max(abs(s-sN));
NN = 1;
while delta > TOL
    a(NN) = trapz(t,s.*cos((NN)*omega).*t);
    b(NN) = trapz(t,s.*sin((NN)*omega).*t);
    sN = sN + a(NN).*cos((NN*omega).*t) + b(NN).*sin((NN*omega).*t);
    delta = max(abs(s-sN));
    dV(NN) = delta;
    NN = NN + 1;
end
figure(1)
semilogy(1:NN-1,dV,'LineWidth',2)
grid on
box on
set(gca,'FontSize',14)
xlabel('Terme in Forurierreihe') ylabel('$\max\vert f_N(t)-f(t)\vert$', 'Interpreter','latex')
figure(2)
hold on
plot(t,s)
plot(t,sN)
hold off
```



Beispiel-Aufgabe 4

Betrachten Sie einen zweidimensionalen Random Walk auf einer Kreisfläche mit dem Radius $R = 1$. Der Random Walker startet im Mittelpunkt des Kreises. Seien die kartesischen Koordinaten des Walkers im n -ten Schritt (x_n, y_n) , dann führt ihn der nächste Schritt zu den Koordinaten

$$x_{n+1} = x_n + \delta \cos(\varphi_n), \quad y_{n+1} = y_n + \delta \sin(\varphi_n)$$

d.h. er springt in die Richtung φ_n mit der Schrittlänge δ . Die Richtung φ_n wird in jedem Schritt zufällig gleichverteilt im Intervall $[0, 2\pi]$ gewählt. Die Schrittlänge bleibt in einem Lauf jeweils fixiert und wird vorgegeben. Der Walker führt so lange Sprünge aus, bis er in einem Sprung den Rand der Kreisscheibe überschreiten würde.

Fertigen Sie ein Matlab Skript an, in dem Sie für die Werte $\delta = \{10^{-3}, 10^{-2}, 10^{-1}\}$ den Walker jeweils R -mal laufen lassen und dabei aufzeichnen, wie viele Sprünge er macht, bis er den Rand der Kreisscheibe überschreiten würde. In Ihrem Skript wird dann für jeden Wert von δ ein Histogramm ausgegeben, welches die Verteilung der Schrittzahlen zeigt. Als weiteren Plot gibt ihr Skript eine Grafik aus, in der die Mittelwerte der Schrittzahlen über δ mittels einer doppel-logarithmischen Skala dargestellt werden. Die Graphiken sind zu formatieren.

```
delta = [10^(-3),10^(-2),10^(-1)];
R = 100;
x = [0;0];
walklaenge = zeros(3,R);
for d=1:length(delta)
    for rr = 1:R
        test = 0;
        x = [0,0];
        while test < 1
            winkel = 2*pi*rand;
            xneu = x + [delta(d)*cos(winkel);delta(d)*sin(winkel)];
            x = xneu;
            test = norm(x);
            walklaenge(d,rr) = walklaenge(d,rr) + 1;
        end
    end
end
figure
for ii = 1:length(delta)
    subplot(1,length(delta),ii)
    histogram(walklaenge(ii,:),21)
    xlabel('Schritte'); title(['\delta = ' num2str(delta(ii))'],'Interpreter','latex')
end
mittelwerte = mean(walklaenge,2);
figure
loglog(delta,mittelwerte,'-o','LineWidth',3,MarkerSize=10)
grid on
xlabel('delta')
ylabel('Mittelwert Schrittzahl')
```

